

Building Reproducible Wi-Fi RF Fingerprinting Pipelines: Signal Collection, Datasets, and Evaluation

Zhen Jia, Jinxiao Zhu, Wenhao Huang, Zewei Guo, Yin Chen

Presented by

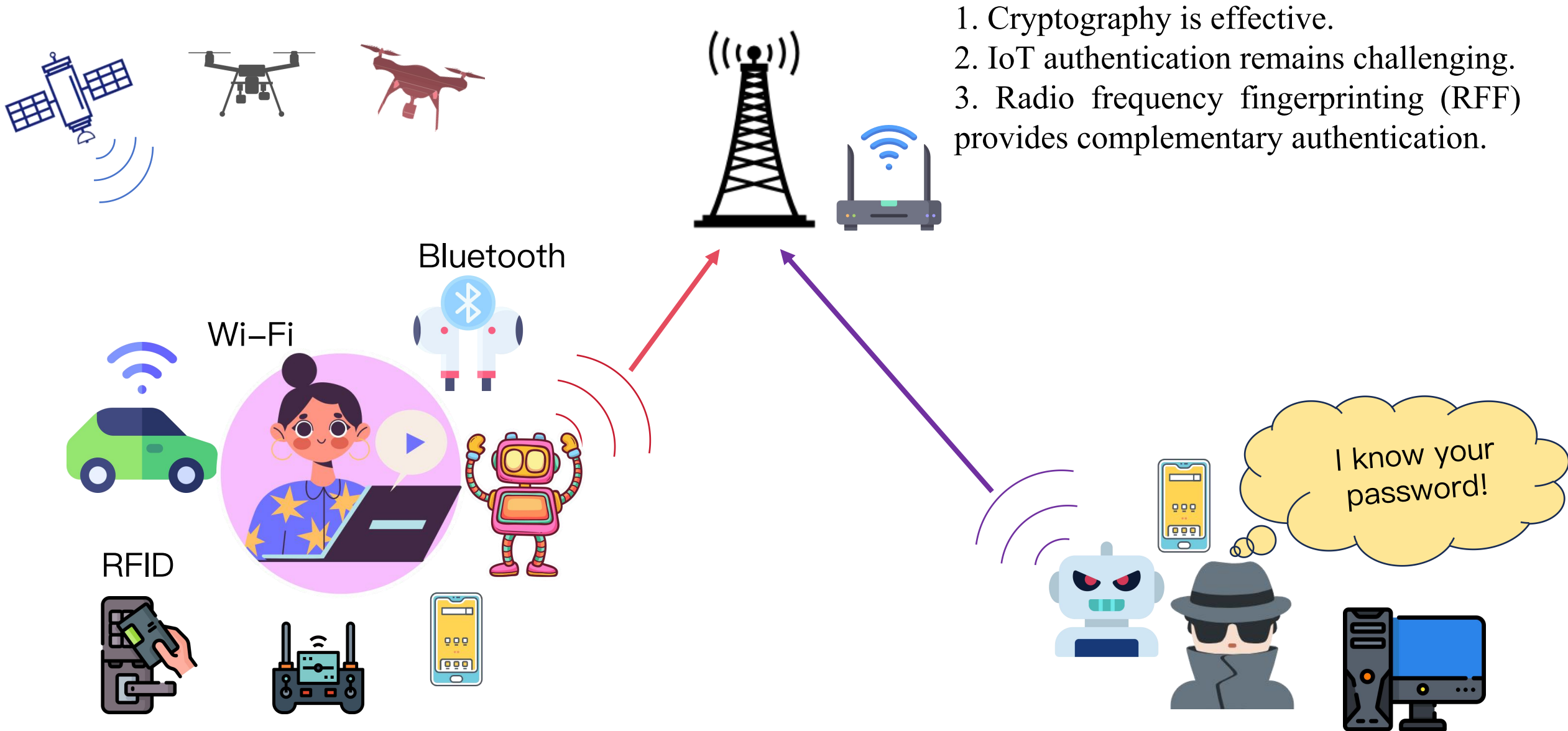
Jinxiao Zhu

2026-10-12

— CONTENTS —

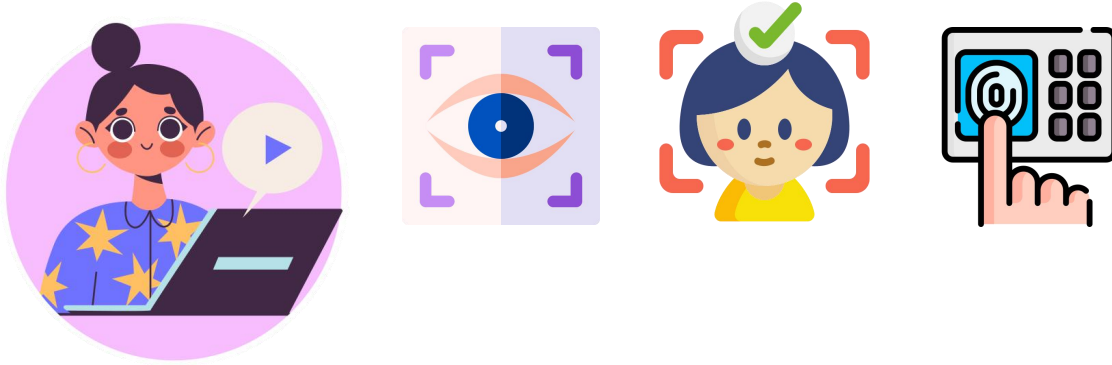
- **1. Background and Task Introduction**
- **2. Wi-Fi Signal and RF Features**
- **3. Data Acquisition Pipeline Walkthrough**
- **4. Lab 1: Load and Explore the Dataset**
- **5. Lab 2: Construct and Visualize RF Features**
- **6. Lab 3: Build and Evaluate the Baseline**
- **7. Summary and Open Discussion**

Authentication in our daily life

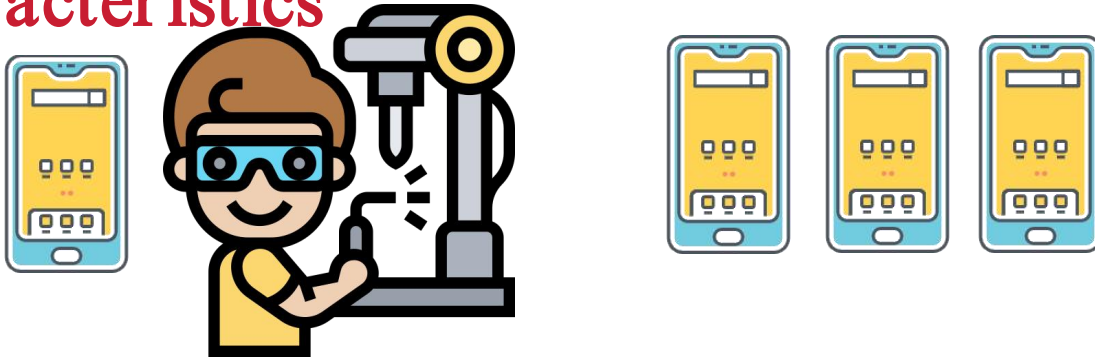


RF fingerprinting (RFF)

Humans have unique biological characteristics

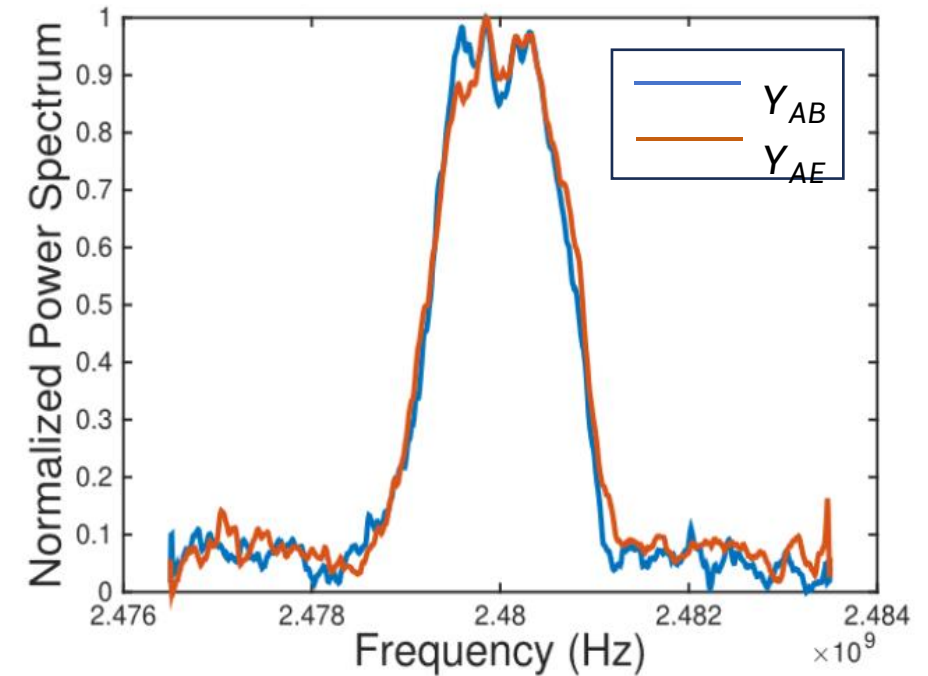
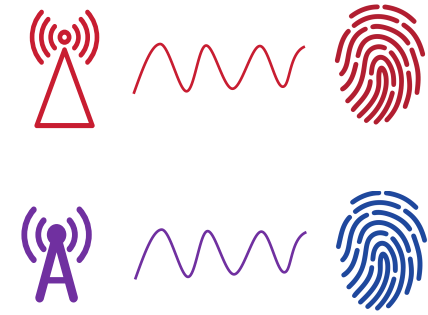


Wireless devices also have unique RF characteristics



Thermal noise,
random vibration,
Hardware non-idealities.

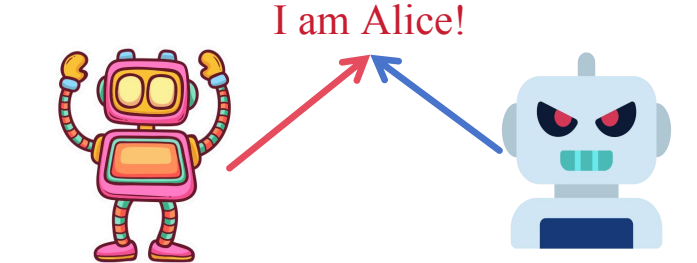
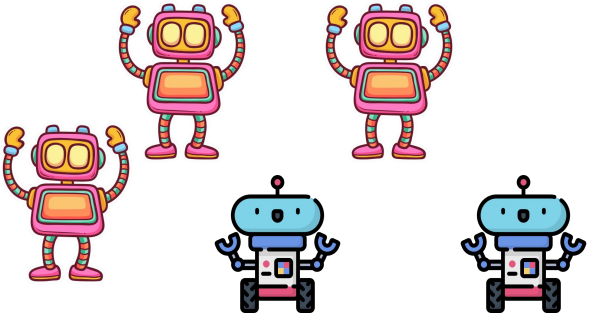
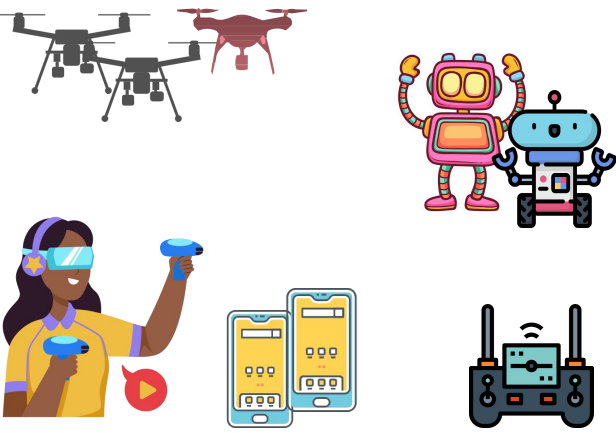
All components are
imperfect.



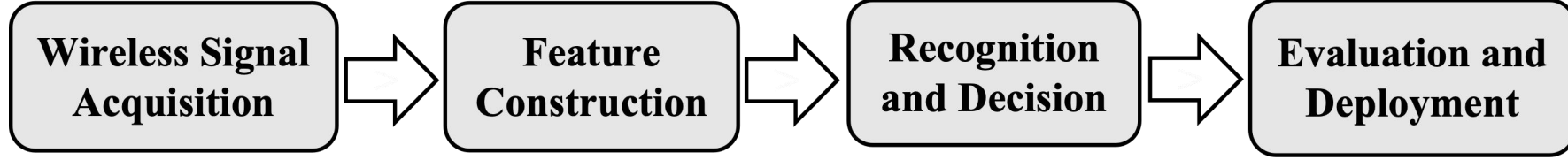
Their signals also have unique
differences.

The applications of RFF

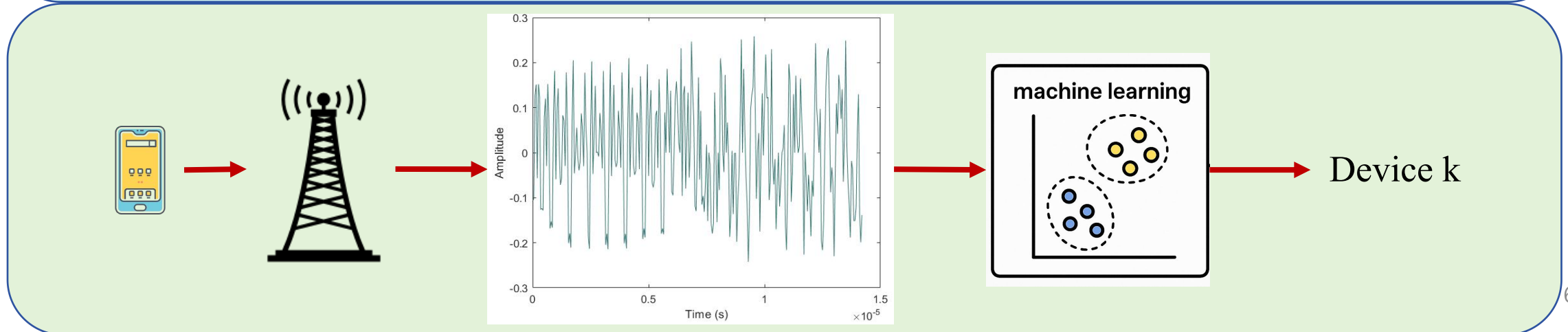
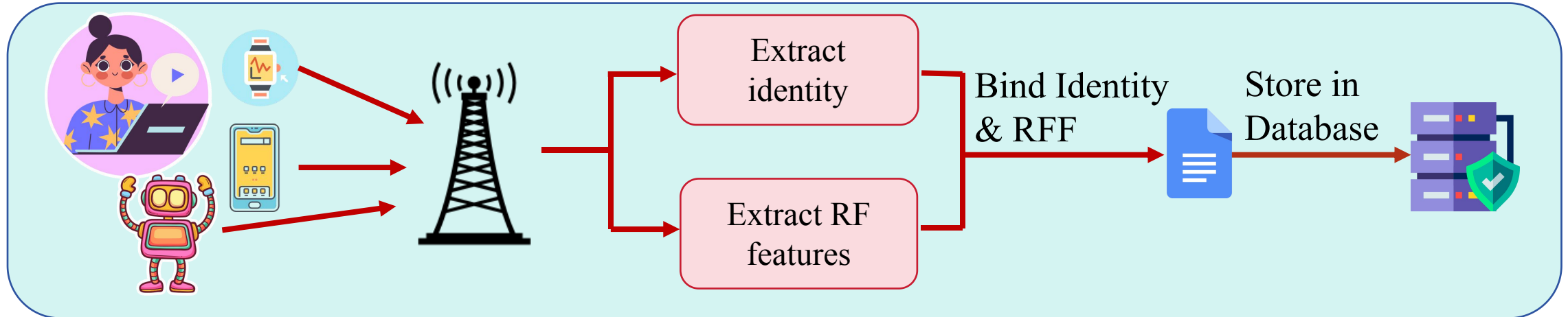
Examples:

Verification	Identification	Classification
 Legitimate transmitter Alice Impersonator Eve		
Is this really Alice?	Which device is transmitting?	What type of device is transmitting?
Accept / Reject	Device ID	Device Type / Model

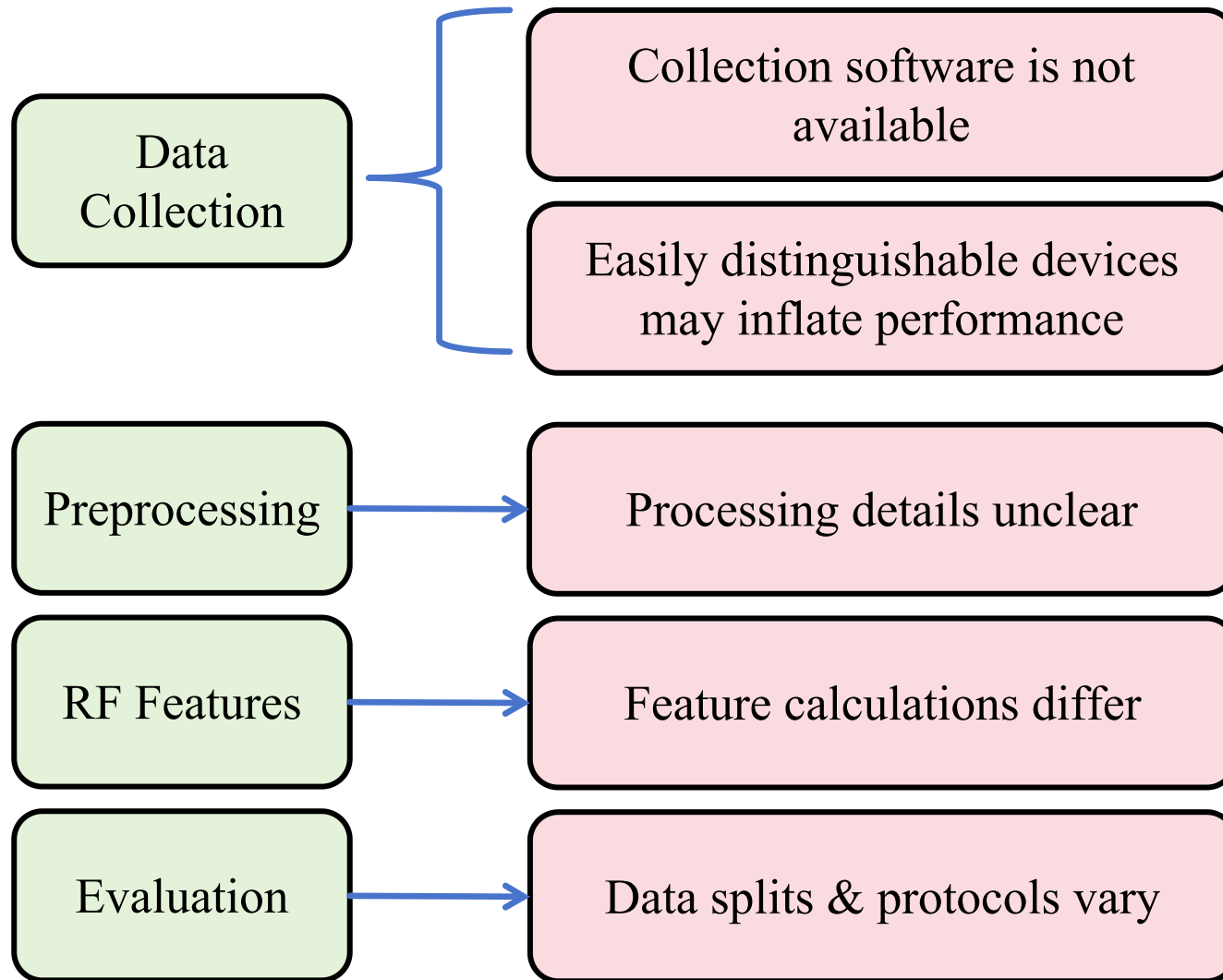
A simplified end-to-end RFF pipeline



Building an RFF experiment requires expertise across wireless systems, signal processing, and machine learning.



Reproducibility Challenges in RFF Research



We need a
reproducible RF
fingerprinting
system!

Learning Objectives

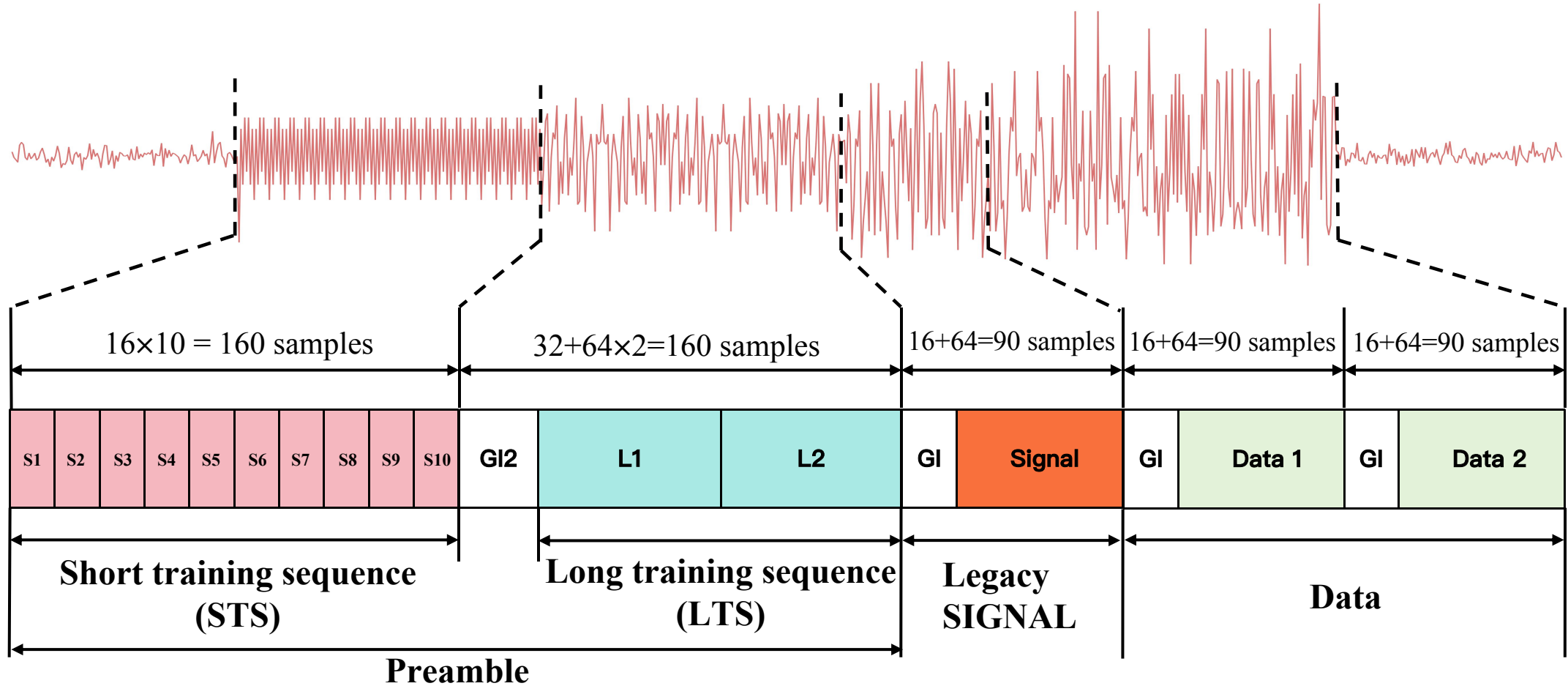
1. Understand the Wi-Fi RFF data acquisition process
2. Build an end-to-end RFF-based identification pipeline
3. Extract and visualize RF features
4. Train and evaluate a baseline model
5. Debug common implementation issues
6. Apply the workflow to your own datasets

— CONTENTS —

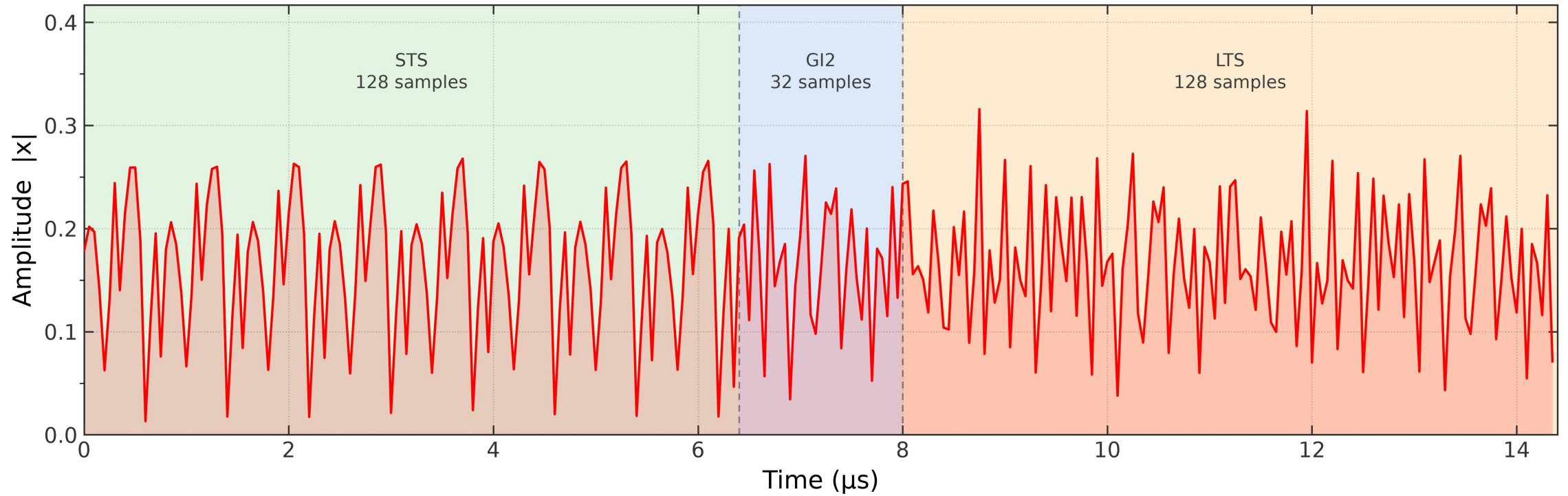
- 1. Background and Task Introduction
- **2. Signal and Feature Introduction**
- 3. Data Acquisition Pipeline Walkthrough
- 4. Lab 1: Load and Explore the Dataset
- 5. Lab 2: Construct and Visualize RF Features
- 6. Lab 3: Build and Evaluate the Baseline
- 7. Summary and Open Discussion

IEEE 802.11g Wi-Fi Signal

IEEE 802.11g 2.4 GHz OFDM Wi-Fi Preamble



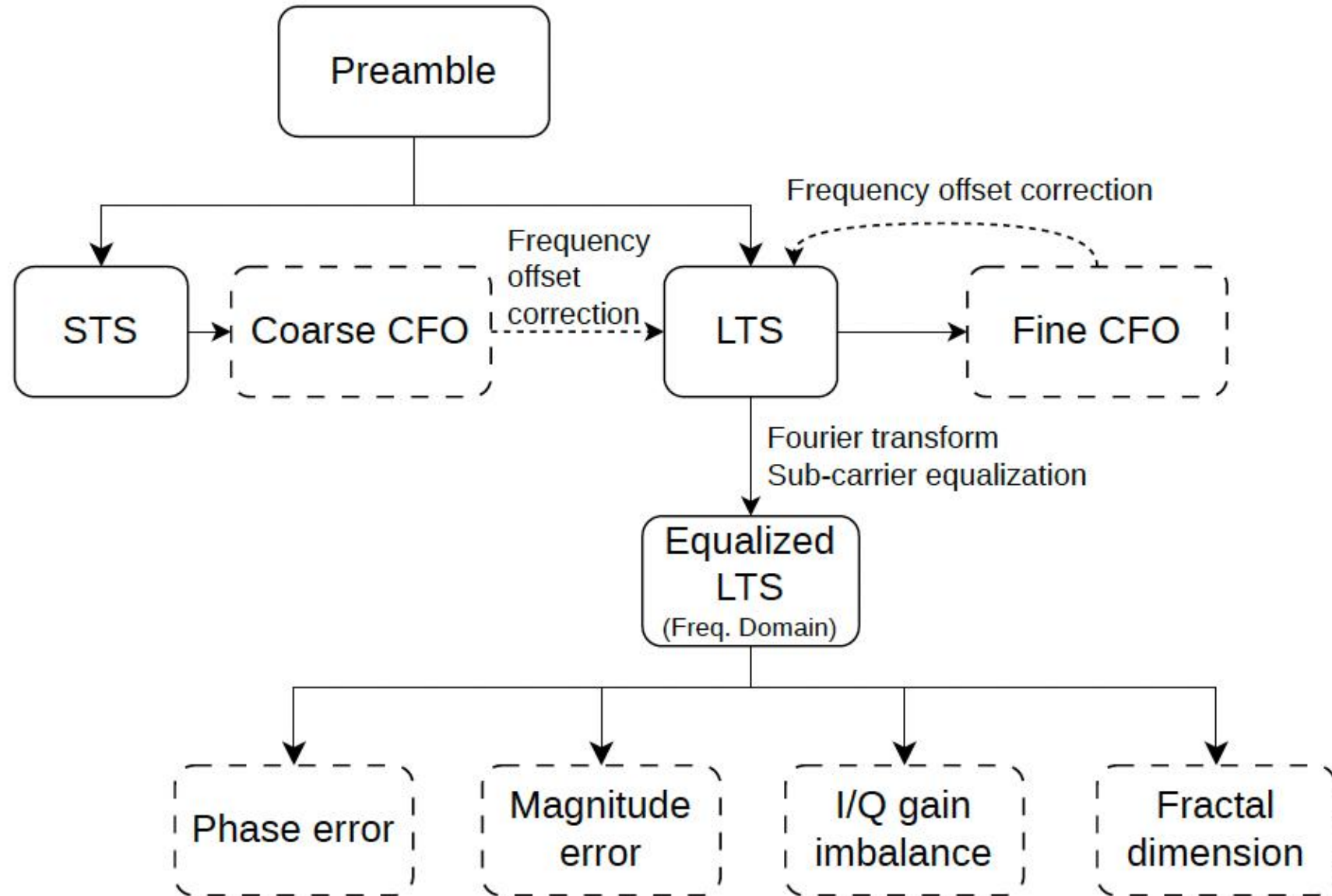
Preamble used in this tutorial



From S3 to S10, 128 samples.
S1 and S2 are dropped.

File:24_d7_eb_38_c7_e8_pre.csv
frame number:119

Feature extraction workflow



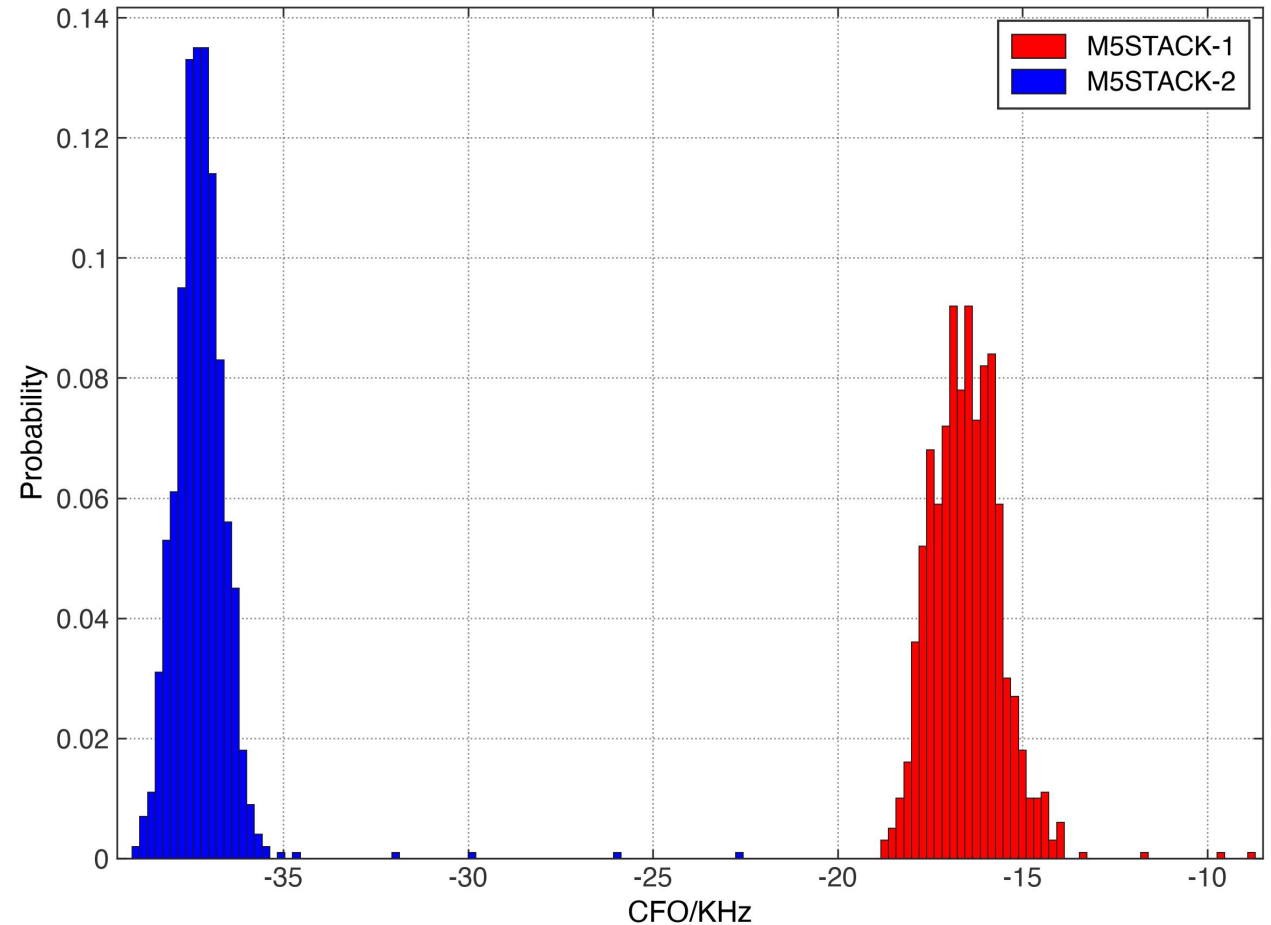
Representative RF Features - CFO

Carrier Frequency Offset (CFO)

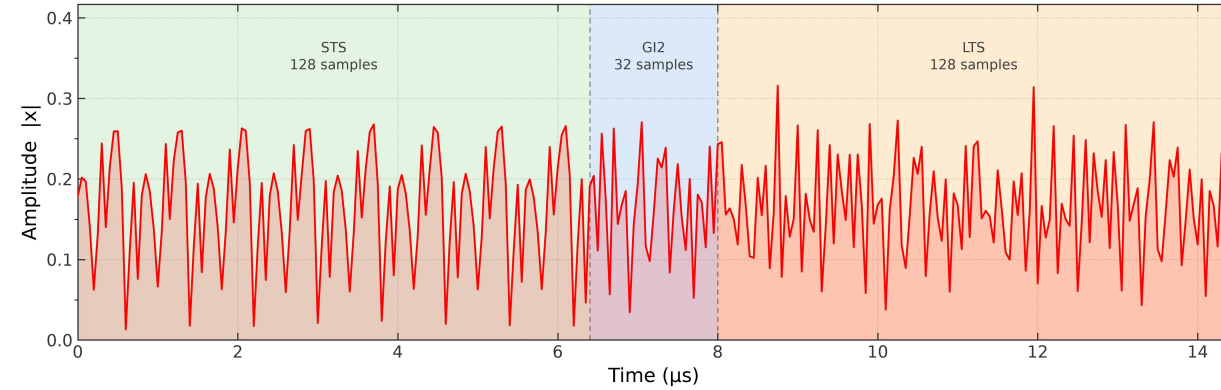
Difference between the received carrier frequency and the expected carrier frequency.

Main sources

1. Oscillator mismatch
2. Doppler shift due to mobility



Representative RF Features - CFO



$$\mathbf{X} = \left[\underbrace{x_1, \dots, x_{128}}_{\text{STS}}, \dots, \underbrace{x_{161}, \dots, x_{288}}_{\text{LTS}} \right].$$

Use STS to calculate Coarse CFO

$$\alpha_S = \frac{1}{16} \angle \left(\sum_{i=1}^{112} \mathbf{x}[i]^\dagger \mathbf{x}[i + 16] \right).$$

Use Coarse CFO to correct LTS

$$\hat{\mathbf{x}}[j] = \mathbf{x}[160 + j] e^{-j(j-1)\alpha_S}, \quad j = 1, 2, \dots, 128.$$

$$j = \sqrt{-1}$$

Use LTS to calculate Fine CFO

$$\alpha_L = \frac{1}{64} \angle \left(\sum_{i=1}^{64} \hat{\mathbf{x}}[i]^\dagger \hat{\mathbf{x}}[i + 64] \right).$$

The total CFO can be obtained as

$$\alpha_F = \alpha_S + \alpha_L.$$

Then, the fine CFO correction can be expressed as

$$\check{\mathbf{x}}[j] = \hat{\mathbf{x}}[j] e^{-j(j-1)\alpha_L}, \quad j = 1, 2, \dots, 128.$$

Here, we focus on the LTS after coarse and fine CFO corrections.

$$\check{\mathbf{x}}_1[m], \quad \check{\mathbf{x}}_2[m], \quad m = 0, 1, \dots, 63.$$

A 64-point FFT converts each CFO-corrected LTS from the time domain to the subcarrier domain.

$$X_n[k] = \sum_{m=0}^{63} \check{x}_n[m] e^{-j2\pi km/64}, \quad n \in \{1, 2\}, \quad k = 0, \dots, 63.$$

The channel response is estimated by averaging the two received LTS symbols and comparing them with the known ideal LTS.

$$\hat{H}[k] = \frac{X_1[k] + X_2[k]}{2L_{\text{ideal}}[k]}. \quad \Rightarrow \quad \text{Channel equalization} \quad I_n[k] = \frac{X_n[k]}{\hat{H}[k]}, \quad n \in \{1, 2\}.$$

After compensating for the channel effect, we further extract the following RF features.

$$I_n[k] \rightarrow \begin{cases} \text{Phase Error} \\ \text{Magnitude Error} \\ \text{I/Q gain imbalance} \\ \text{Fractal Dimension} \end{cases}$$

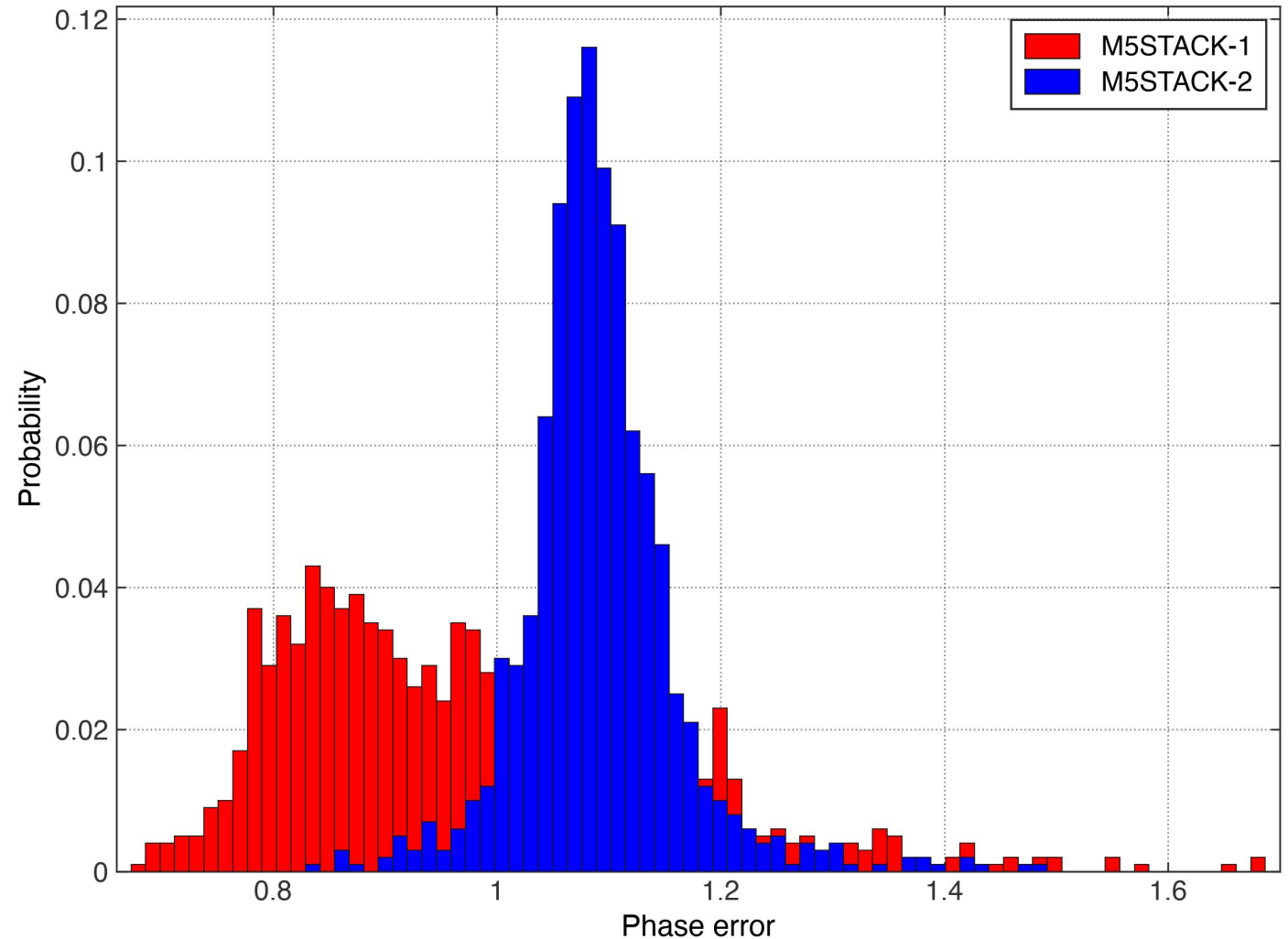
Representative RF Features – Phase Error

Phase error

Phase difference between the equalized and reference LTS

Main sources

- Residual CFO
- Oscillator phase noise
- Synchronization errors
- Other RF impairments.



$$\mathbf{p}_n[k] = \angle \mathbf{l}_n[k] - \angle \mathbf{l}_{\text{ideal}}[k], n \in \{1, 2\}, k = 1, 2, \dots, 64$$

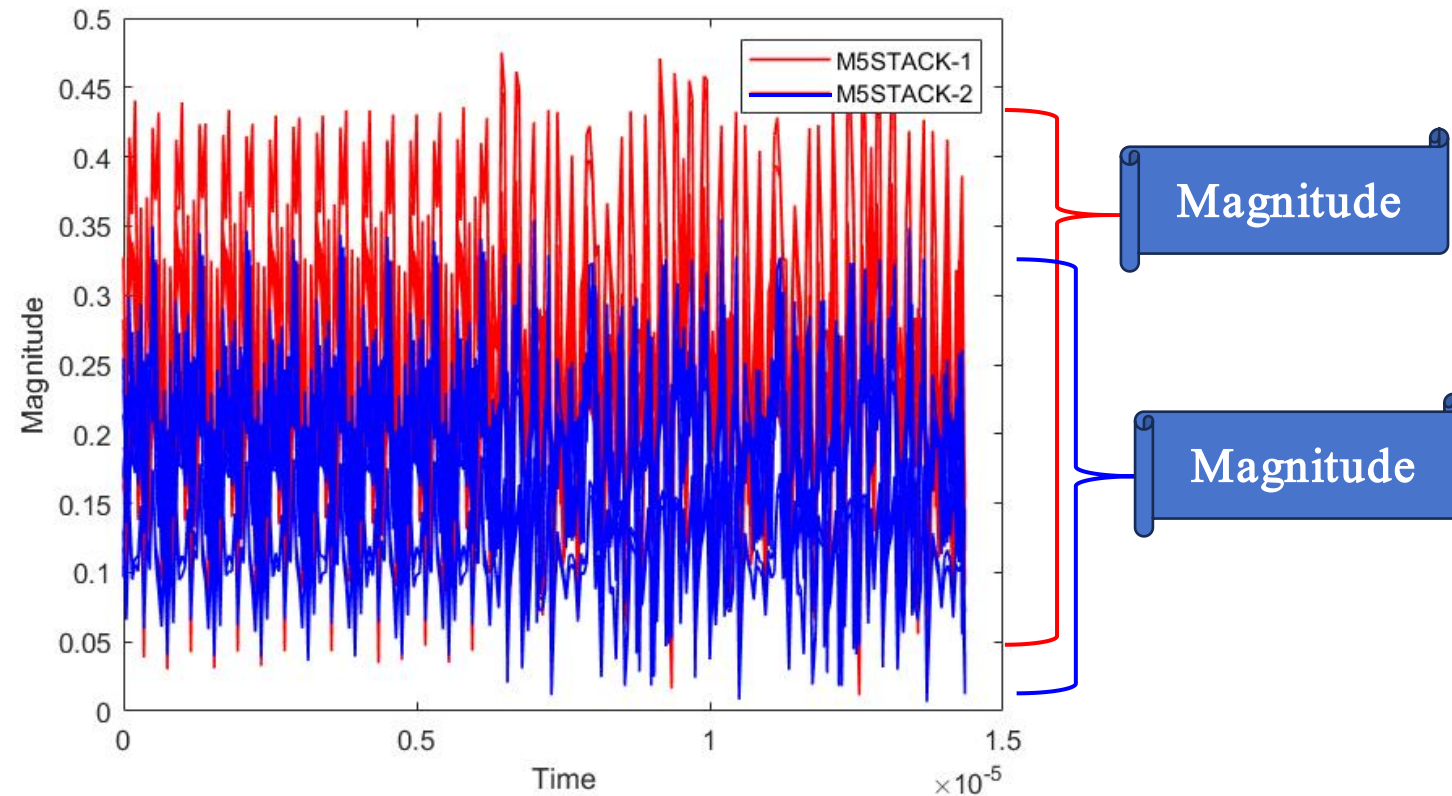
Representative RF Features – Magnitude Error

Magnitude error

Magnitude difference between the equalized and reference LTS

Main sources

- Power-amplifier characteristics
- RF front-end imperfections.



$$\mathbf{m}_n[k] = |\mathbf{l}_n[k]| - |\mathbf{l}_{\text{ideal}}[k]|, n \in \{1, 2\}, k = 1, 2, \dots, 64$$

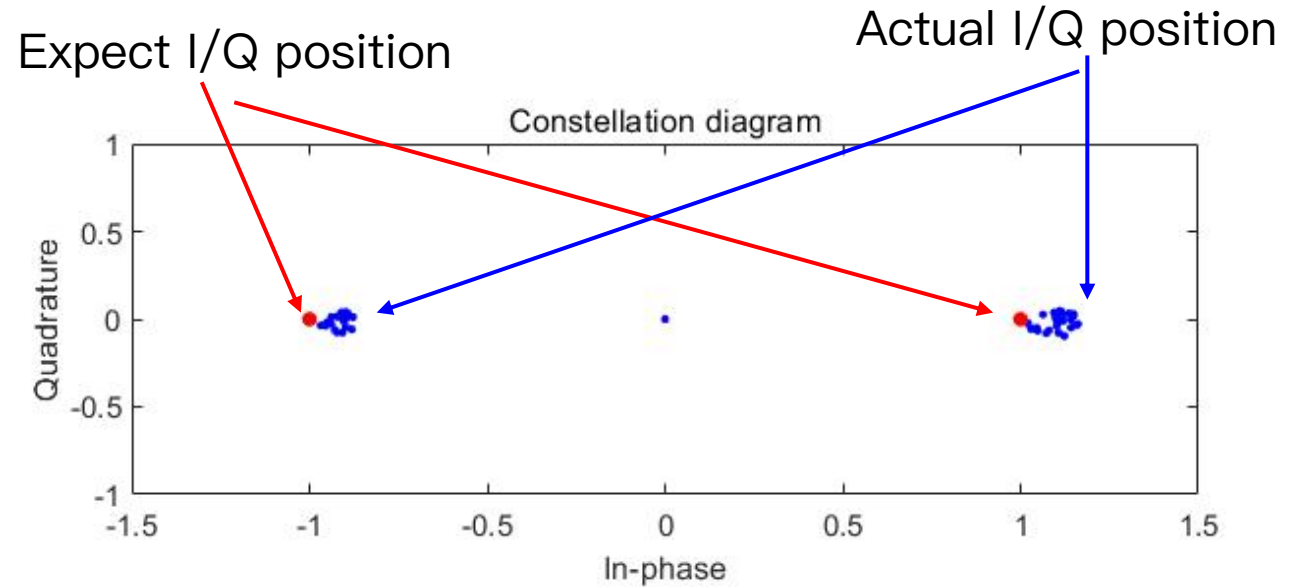
Representative RF Features - IQI

I/Q gain imbalance

Energy imbalance between the I and Q components

$$IQI_n = \frac{\|\Re\{\mathbf{I}_n\}\|^2}{\|\Im\{\mathbf{I}_n\}\|^2}, n \in \{1, 2\}$$

- Ideal balance: $IQI \approx 1$



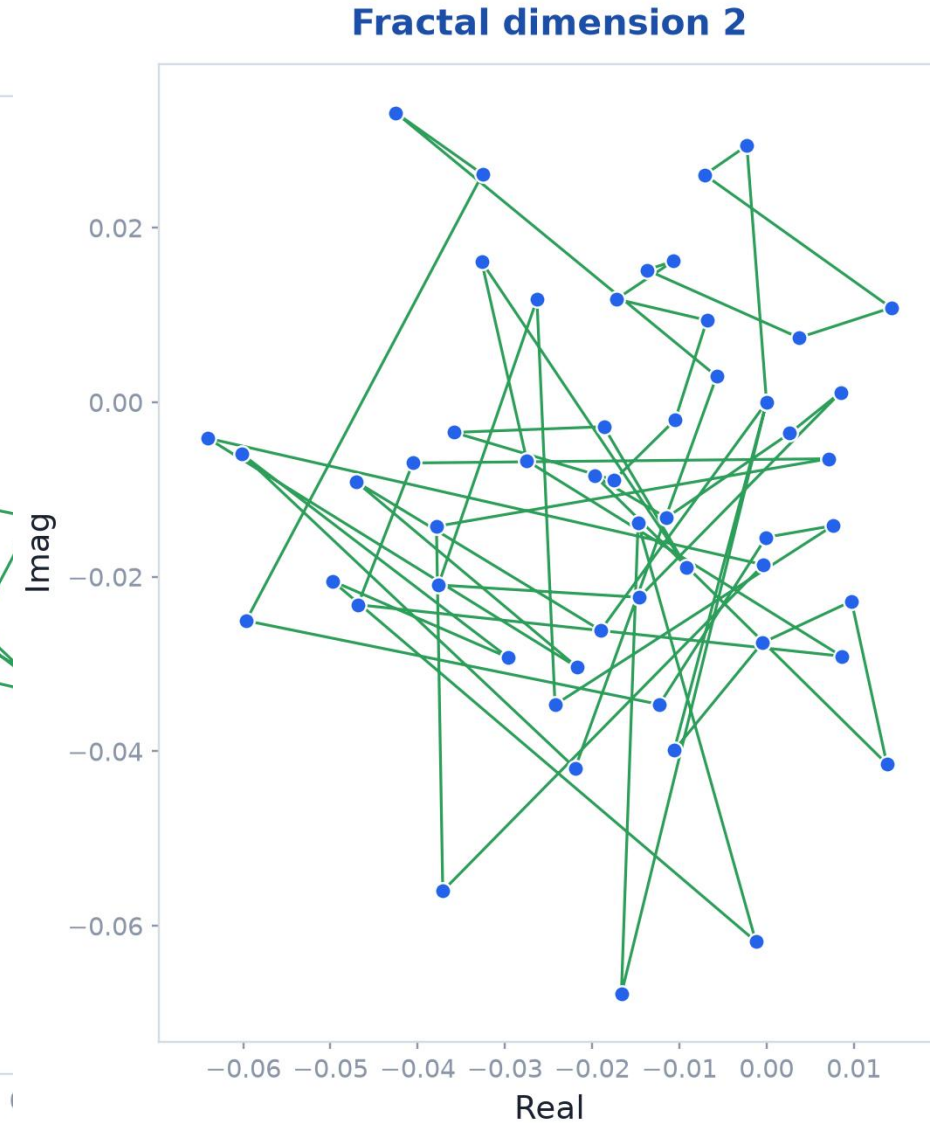
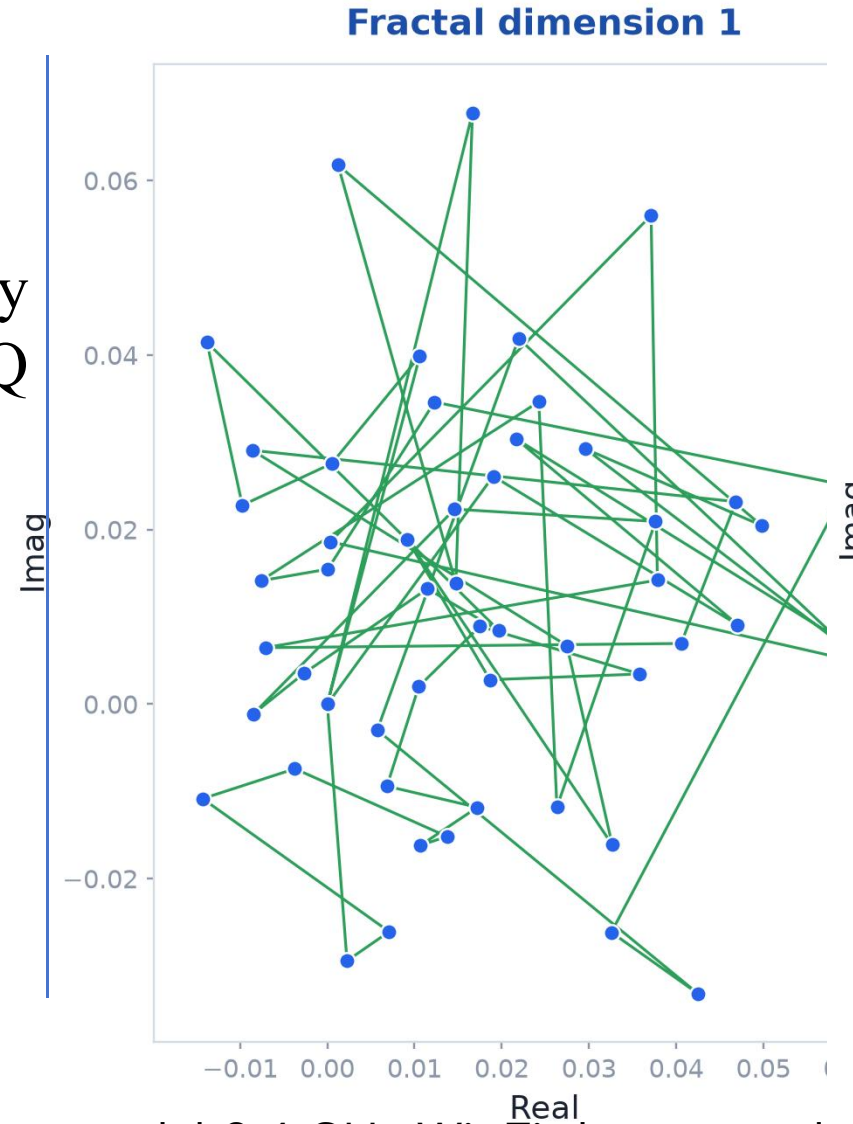
Representative RF Features – Fractal Dimension

Fractal dimension

Quantifies the complexity and self-similarity of the I/Q signal distribution

Interpretation

Different I/Q structures → different fractal dimensions



Ref.:

- SMOFFI: A large-scale same-model 2.4 GHz Wi-Fi dataset and reproducible framework for RF fingerprinting, Computer Networks, Volume 282, 2026.

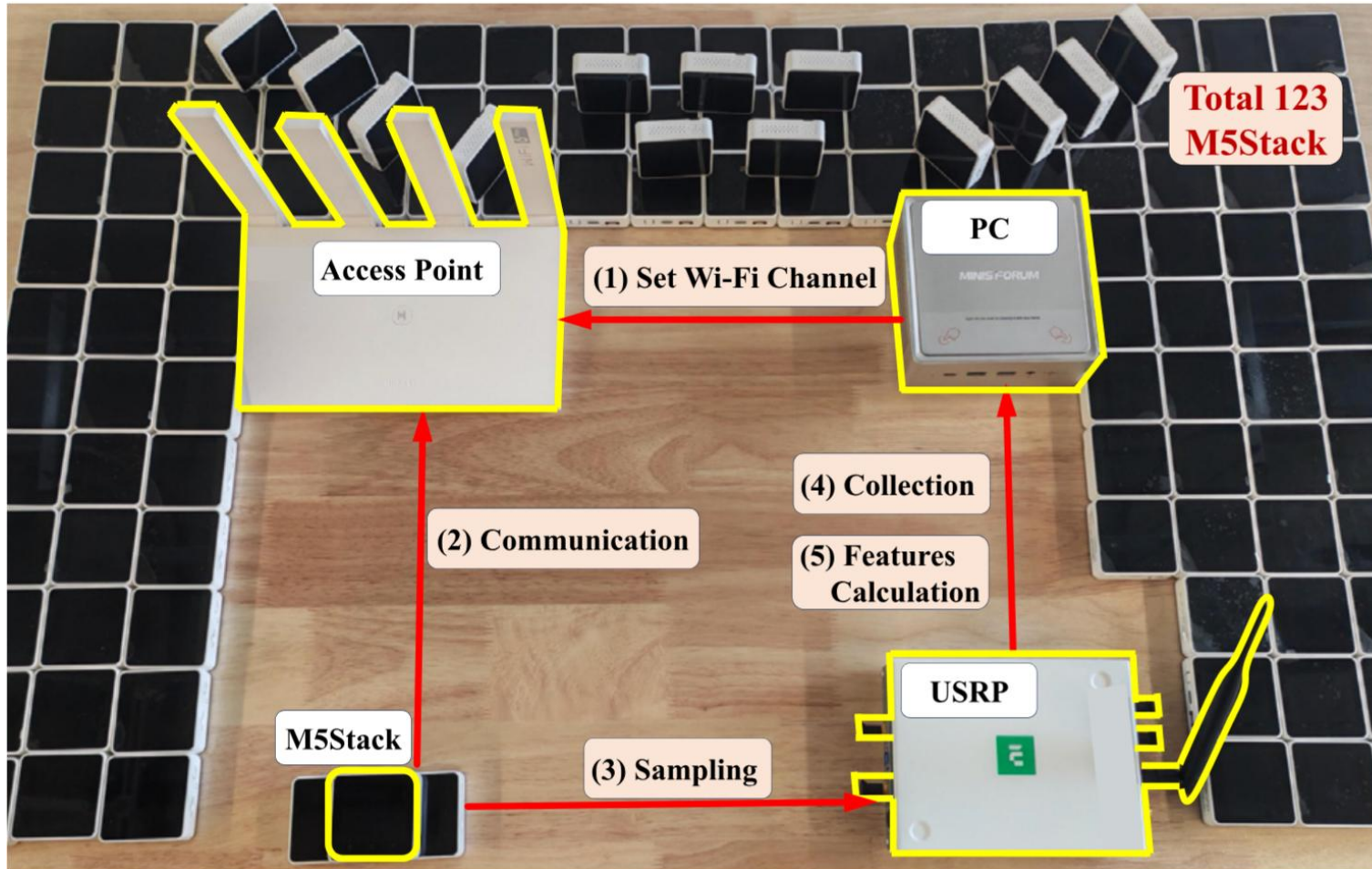
Representative RF Features – Summary

- **CFO** → frequency offset
- **Phase Error** → phase deviation
- **Magnitude Error** → magnitude deviation
- **I/Q Gain Imbalance** → I/Q energy imbalance
- **Fractal Dimension** → I/Q distribution complexity/self-similarity

— CONTENTS —

- 1. Background and Task Introduction
- 2. Signal and Feature Introduction
- **3. Data Acquisition Pipeline Walkthrough**
- 4. Lab 1: Load and Explore the Dataset
- 5. Lab 2: Construct and Visualize RF Features
- 6. Lab 3: Build and Evaluate the Baseline
- 7. Summary and Open Discussion

Data collection system



Transmitter: M5Stack Core2

Receiver: USRP B210

Access Point: Huawei WS7100 V2

Processor PC: Minisforum NPB7

— CONTENTS —

- 1. Background and Task Introduction
- 2. Signal and Feature Introduction
- 3. Data Acquisition Pipeline Walkthrough
- **4. Lab 1: Load and Explore the Dataset**
- 5. Lab 2: Construct and Visualize RF Features
- 6. Lab 3: Build and Evaluate the Baseline
- 7. Summary and Open Discussion

Dataset 1 – Raw Preamble/IQ Data

- Repository name: RFFI-IQ_only-wifi-802.11g-2.4G-123-m5stack Direct URL to data <https://www.kaggle.com/datasets/yinchen1986/rffi-123-m5stack-iq-wifi-802-11g-2-4g>

RFFI-IQ_only-wifi-802.11g-2.4G-123-m5stack



Data Card Code (2) Discussion (0) Suggestions (0)

About Dataset

This dataset was acquired using 1 NI B-\$210\$ receiver operated via GNU Radio, along with 123 M5Stack Core2 transmitters, 1 Huawei WS700 V2 Wi-Fi access point, and 1 Minisforum NAB6 Lite computer. 1,000 signal frames were collected from each transmitter over a 5-minute interval. All transmitters and the NI B-210 receiver operated based on the IEEE 802.11g standard with a bandwidth of 20\$ MHz. The receiver operated via IEEE 802.11 a/g/p transceiver model with 20 MS/s sampling rate. Specially, each recorded frame mainly contains two elements: (1) MAC address of the device and (2) Raw samples of preamble.

Usability ⓘ

4.71

License

MIT

Expected update frequency

Not specified

Tags

Data Explorer

952.05 MB

- 24_d7_eb_38_c7_e8_pre.csv
- 24_d7_eb_38_c7_fc_pre.csv
- 24_d7_eb_38_c8_d8_pre.csv
- 24_d7_eb_38_cc_60_pre.csv
- 24_d7_eb_38_cc_e4_pre.csv
- 24_d7_eb_38_cc_f0_pre.csv
- 24_d7_eb_38_cd_00_pre.csv
- 24_d7_eb_38_cd_5c_pre.csv
- 24_d7_eb_38_cf_ec_pre.csv
- 24_d7_eb_38_d0_b0_pre.csv
- 24_d7_eb_38_d0_cc_pre.csv
- 24_d7_eb_38_d6_c4_pre.csv
- 24_d7_eb_38_d8_64_pre.csv
- 24_d7_eb_38_da_68_pre.csv
- 24_d7_eb_38_de_1c_pre.csv
- 24_d7_eb_38_de_dc_pre.csv
- 24_d7_eb_38_e0_a4_pre.csv
- 24_d7_eb_38_e0_dc_pre.csv
- 24_d7_eb_38_e1_04_pre.csv
- 24_d7_eb_38_e1_08_pre.csv

24_d7_eb_38_c7_e8_pre.csv (7.86 MB)

Detail Compact Column

3 of 3 columns

Suggest Edits

# Device Number	MAC_address	preamble
1	1 unique value	1000 unique values
1	24:d7:eb:38:c7:e8	[0.21494181+1.12491198e-01j -0.187788035+5.97857460e-02j -0.19699693-3.15866588e-02j -0.04773894+3....
1	24:d7:eb:38:c7:e8	[6.95821829e-

Dataset 2 - Extracted RF Features + IQ Data

- Repository name: RFFI-kf_feature-IQ-wifi-802.11g-2.4G-123-m5stack Direct URL to data <https://www.kaggle.com/datasets/yinchen1986/rffi-kf-feature-iq-wifi-802-11g-2-4g-123-m5stack>

MAC_address	preamble	iq_preamble	CFO	short_freq	long_freq	frac_dimension_1	frac_dimension_2	iqi_1	iqi_2	mag_error_vec_1	mag_error_mean_1	mag_error_var_1	mag_error_vec_2	mag_error_mean_2	mag_error_var_2	phase_error_vec_1	phase_error_mean_1	phase_error_var_1	phase_error_vec_2	phase_error_mean_2	phase_error_var_2
24:d7:eb:38:c7:e8	[0.2149418, 1+1.12491, 198e-01j - 0.1877803, 5+5.97857, 460e-02j -	[0+0i, 0+0i, 0+0i, 0+0i, 0.056486-0.0012075i, 0.056201+0.0067885i, 0.051797+0	-13449	0.0076333	-0.01186	1.94521183	1.94521183	3.1	3.1	[0, 0, 0, 0, 0, 0, 0.9435, 0.94339, 0.93088, 0.94781, 0.93304, 0.94134,	0.7096838	0.1214105	[0, 0, 0, 0, 0, 0, 0.9435, 0.94339, 0.93088, 0.94781, 0.93304, 0.94134,	0.7096838	0.1214105	[0, 0, 0, 0, 0, 0.021373, -0.12021, 2.4179, 2.3975, -0.51497, -0.47713, 2.4487, -	1.01628851	2.60134485	[0, 0, 0, 0, 0, 0, -3.1202, 3.0214, 5.5595, 5.539, 2.6266, 2.6645, 5.5903,	1.21263805	6.77103655
24:d7:eb:38:c7:e8	[6.9582182, 9e-03+0.0980, 8647j - 1.1072112, 6e-01- 0.0075075	[0+0i, 0+0i, 0+0i, 0+0i, 0.0001158-0.015442i, 0.010314-0.025965i, 0.0011546-	-14333	0.0059857	-0.01049	1.98460317	1.98460317	6.82	6.82	[0, 0, 0, 0, 0, 0, 0.98456, 0.97206, 0.97037, 0.97565, 0.98116, 0.98059,	0.7711075	0.1403295	[0, 0, 0, 0, 0, 0, 0.98456, 0.97206, 0.97037, 0.97565, 0.98116, 0.98059,	0.7711075	0.1403295	[0, 0, 0, 0, 0, 0, 1.5633, 1.1927, 4.6734, 4.4511, 1.943, 1.8193, 2.1477,	1.25222033	2.99540827	[0, 0, 0, 0, 0, 0, -1.5783, -1.9489, 1.5318, 1.3095, -1.1986, -1.3223, 5.2893, -	0.17229786	6.54503861
24:d7:eb:38:c7:e8	[0.1992858, 2- 1.1496319, 6e-01j - 0.0513321, 2+2.00689, 673e-01i	[0+0i, 0+0i, 0+0i, 0+0i, 0.049884+0.017883i, 0.042601+0.013117i, 0.047344+0	-16640	0.0071055	-0.01233	1.94990715	1.94990715	2.57	2.57	[0, 0, 0, 0, 0, 0, 0.94701, 0.95542, 0.94766, 0.93026, 0.9381, 0.95667,	0.7104552	0.1211758	[0, 0, 0, 0, 0, 0, 0.94701, 0.95542, 0.94766, 0.93026, 0.9381, 0.95667,	0.7104552	0.1211758	[0, 0, 0, 0, 0, 0, -0.34421, -0.2987, 2.7011, 2.8285, -0.37563, -0.22416, 2.4299, -	1.00703256	2.68745317	[0, 0, 0, 0, 0, 0, 2.7974, 2.8429, 5.8427, 5.9701, 2.766, 2.9174, 5.5715,	1.30155687	6.05799301

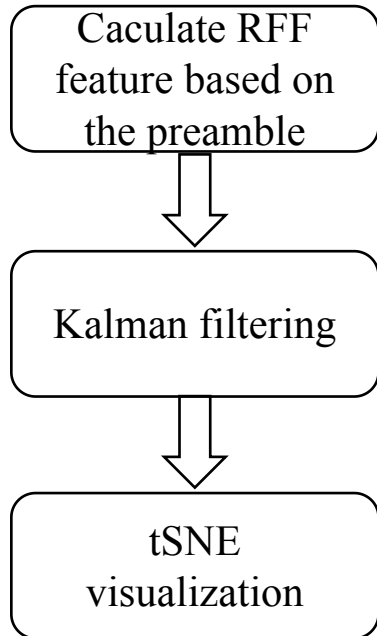
— CONTENTS —

- 1. Background and Task Introduction
- 2. Signal and Feature Introduction
- 3. Data Acquisition Pipeline Walkthrough
- 4. Lab 1: Load and Explore the Dataset
- **5. Lab 2: Construct and Visualize RF Features**
- 6. Lab 3: Build and Evaluate the Baseline
- 7. Summary and Open Discussion

Construct RF features: Caculate RFF feature based on the preamble

- Repository name: RFF_Data_Calculate

<https://www.kaggle.com/code/zeweiguo/rff-data-calculate>



ZEWEI GUO · 6MO AGO · 353 VIEWS

RFF_Data_Calculate

Notebook Input Output Logs Comments (0)

Cite XXXXXXXXXXXXX

Caculate RFF Feature Based on the Preamble

This section extracts ten representative radio-frequency fingerprinting (RFF) features from IEEE 802.11 preamble data. The extracted features include the raw preamble samples, long-preamble I/Q samples, multiple CFO estimations (total, STS-based, and LTS-based), phase and magnitude error metrics, I/Q gain imbalance, and fractal-dimension-based descriptors.

In [1]:

```
import os
import math
import ast
import re
from pathlib import Path
import numpy as np
import pandas as pd

# ----- Utility Functions -----

def distance(a: float, b: float, c: float, d: float) -> float:
    """Euclidean distance between (a,b) and (c,d)."""
    return math.hypot(a - c, d - b)

def power(x: np.ndarray) -> float:
    """Sum of squares."""
    x = np.asarray(x, dtype=float)
```

Version 14 of 14

Runtime: 2h 36m 5s

Input

DATASETS

- RFFI-feature_IQ-wifi-802.11g-2.4G-12
- RFFI-IQ_only-wifi-802.11g-2.4G-123-
- RFFI-kf_feature_IQ-wifi-802.11g-2.4G

Language: Python

Table of Contents

- Caculate RFF Feature Based on the ...
- Kalman Filtering
- tSNE
- Random Forest

Pre-loaded
datasets

Construct RF features: Caculate RFF feature based on the preamble

Caculate RFF Feature Based on the Preamble

This section extracts ten representative radio-frequency fingerprinting (RFF) features from IEEE 802.11 preamble data. The extracted features include the raw preamble samples, long-preamble I/Q samples, multiple CFO estimations (total, STS-based, and LTS-based), phase and magnitude error metrics, I/Q gain imbalance, and fractal-dimension-based descriptors.

Click to
run.

+ Code + Markdown

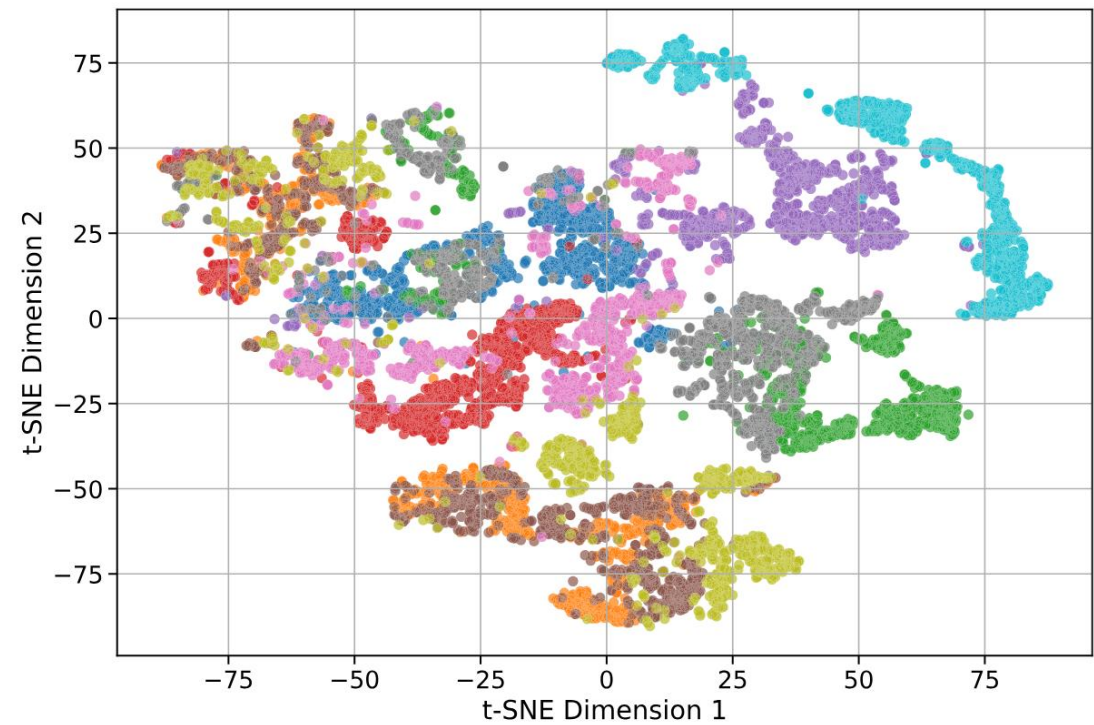
```
import os
import math
import ast
import re
from pathlib import Path
import numpy as np
import pandas as pd

# ----- Utility Functions -----

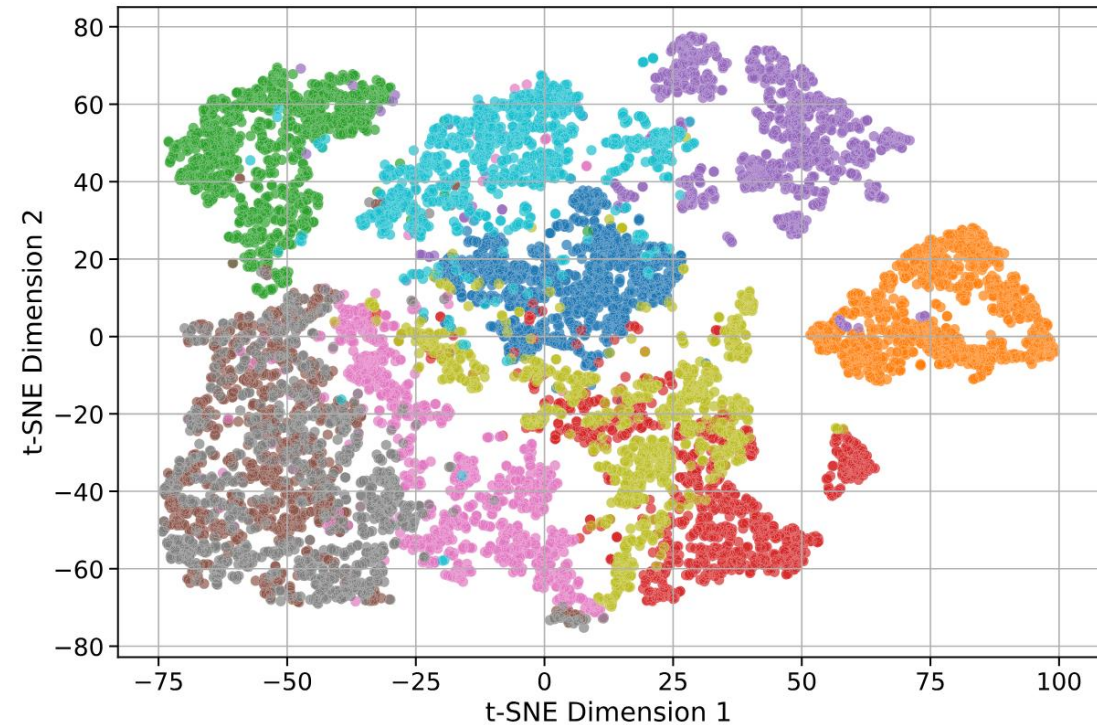
def distance(a: float, b: float, c: float, d: float) -> float:
    """Euclidean distance between (a,b) and (c,d)."""
    return math.hypot(a - c, d - b)

def power(x: np.ndarray) -> float:
    """Sum of squares."""
    x = np.asarray(x, dtype=float)
    return float(np.sum(x * x))
```

Feature visualization



Before Kalman Filtering



After Kalman Filtering

Construct RF features: Kalman filtering

Kalman filtering recursively combines state prediction and noisy observations to obtain a smoother signal estimate.

Kalman Filtering

(You can skip this step and use the pre-processed dataset named "rffi-kf-feature-iq-wifi-802-11g-2-4g-123-m5stack," which allows you to directly perform t-SNE visualization and Random Forest classification.) In this section, the RF features generated in the previous step are refined using Kalman Filtering to improve data smoothness and suppress measurement noise.

Click to
run.



```
import os
import pandas as pd
import numpy as np
try:
    from pykalman import KalmanFilter
except ImportError:
    !pip install pykalman
    from pykalman import KalmanFilter

# Define the Kalman filter function
def apply_kalman_filter(df, columns):
    filtered_data = df.copy()
    for col in columns:
        if col in df.columns:
            try:
```

— CONTENTS —

- 1. Background and Task Introduction
- 2. Signal and Feature Introduction
- 3. Data Acquisition Pipeline Walkthrough
- 4. Lab 1: Load and Explore the Dataset
- 5. Lab 2: Construct and Visualize RF Features
- **6. Lab 3: Build and Evaluate the Baseline**
- 7. Summary and Open Discussion

Random Forest

Random Forest is an ensemble learning method that combines multiple decision trees. Each tree is trained on a randomly sampled subset of the training data and features, and the final class is determined by majority voting.

Random Forest

In this section, a Random Forest classifier is constructed, and its identification performance is evaluated on the above two datasets.

Click to run.

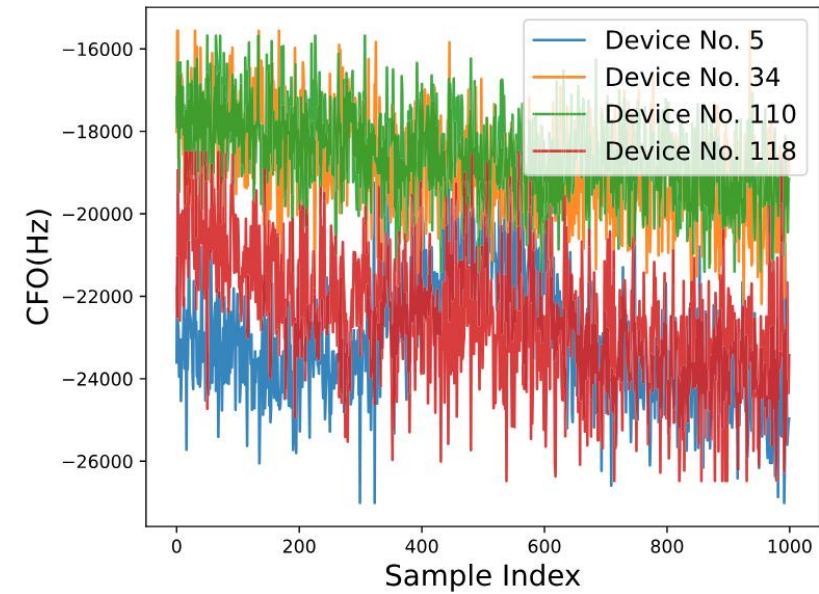
```
import os
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
from mpl_toolkits.axes_grid1.inset_locator import inset_axes
import seaborn as sns

plt.rcParams.update({'font.size': 12})
sns.set_context("notebook", font_scale=1.5)
```

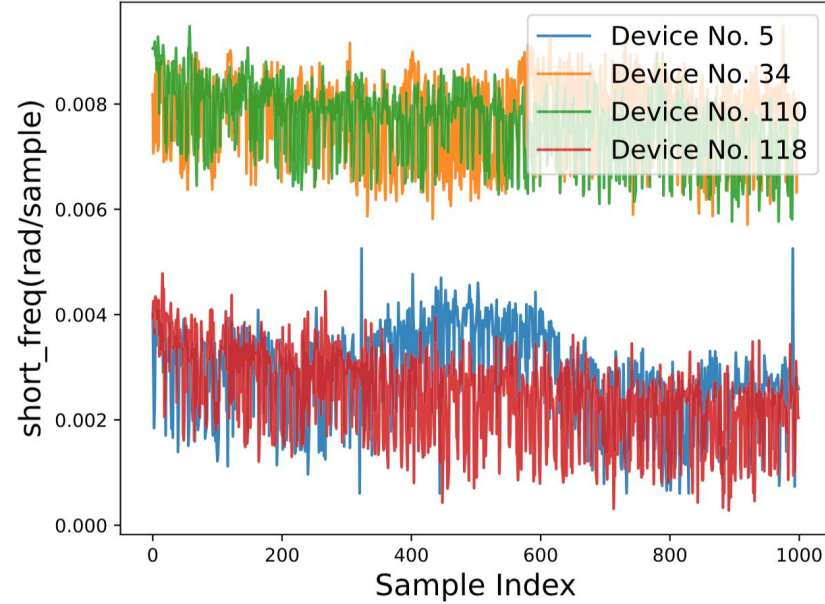
Evaluation results for random forest model.

Dataset & Setting	Accuracy (ACC)
5-Fold Average without KF	82.0
5-Fold Average with KF	88.6

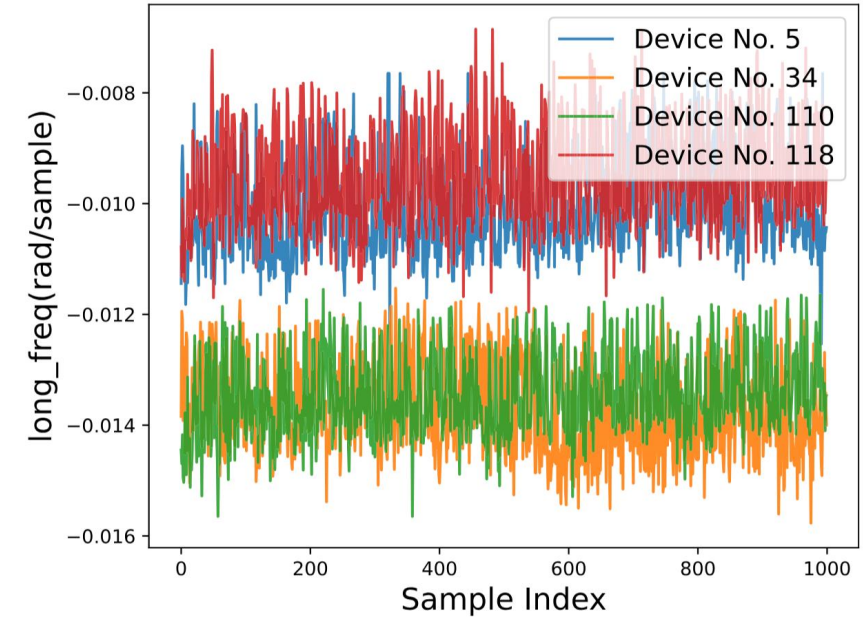
Impact of sessions on representative RF features



(a) CFO



(b) Coarse CFO of STS



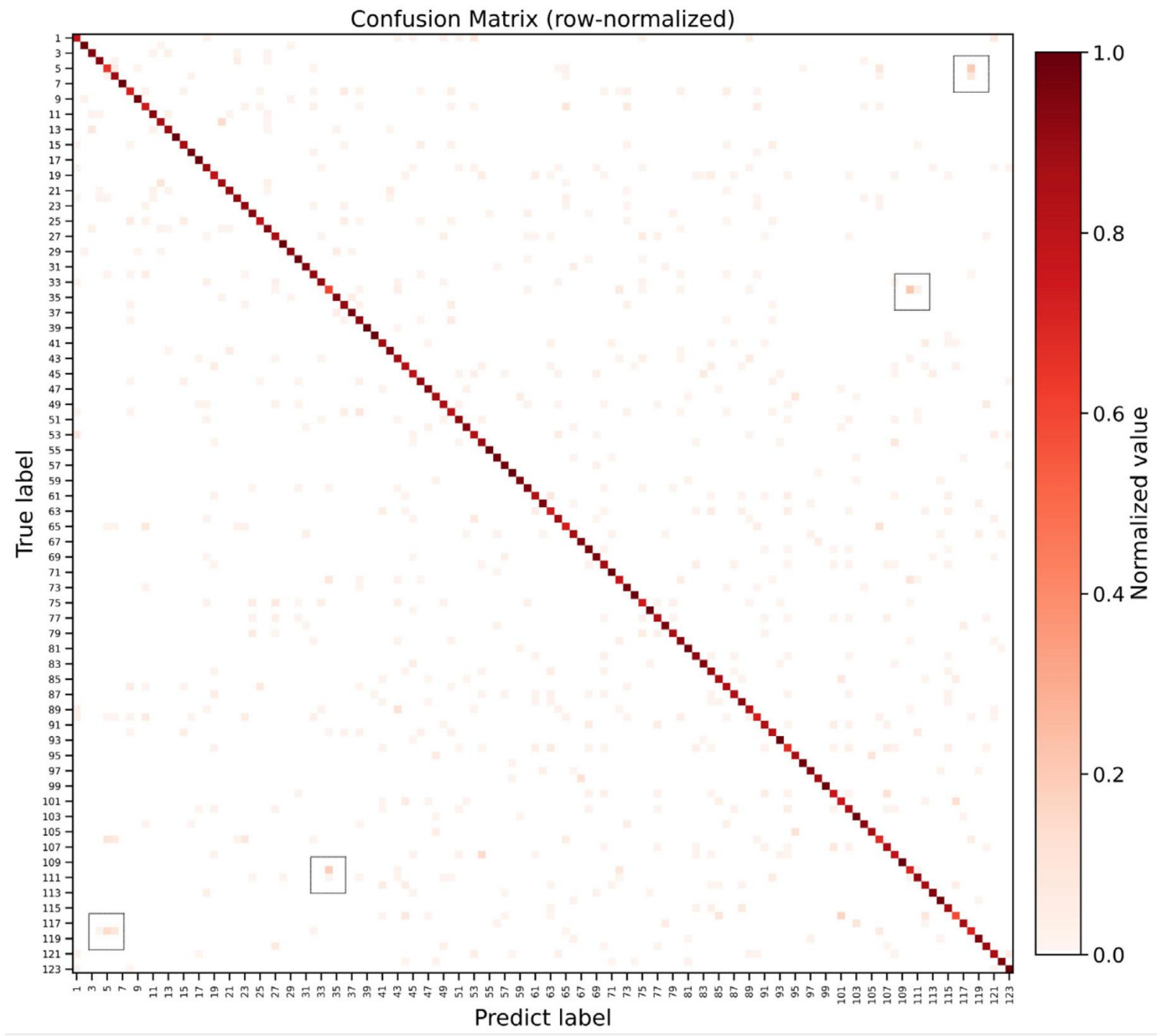
(c) Fine CFO of LTS

Feature importance

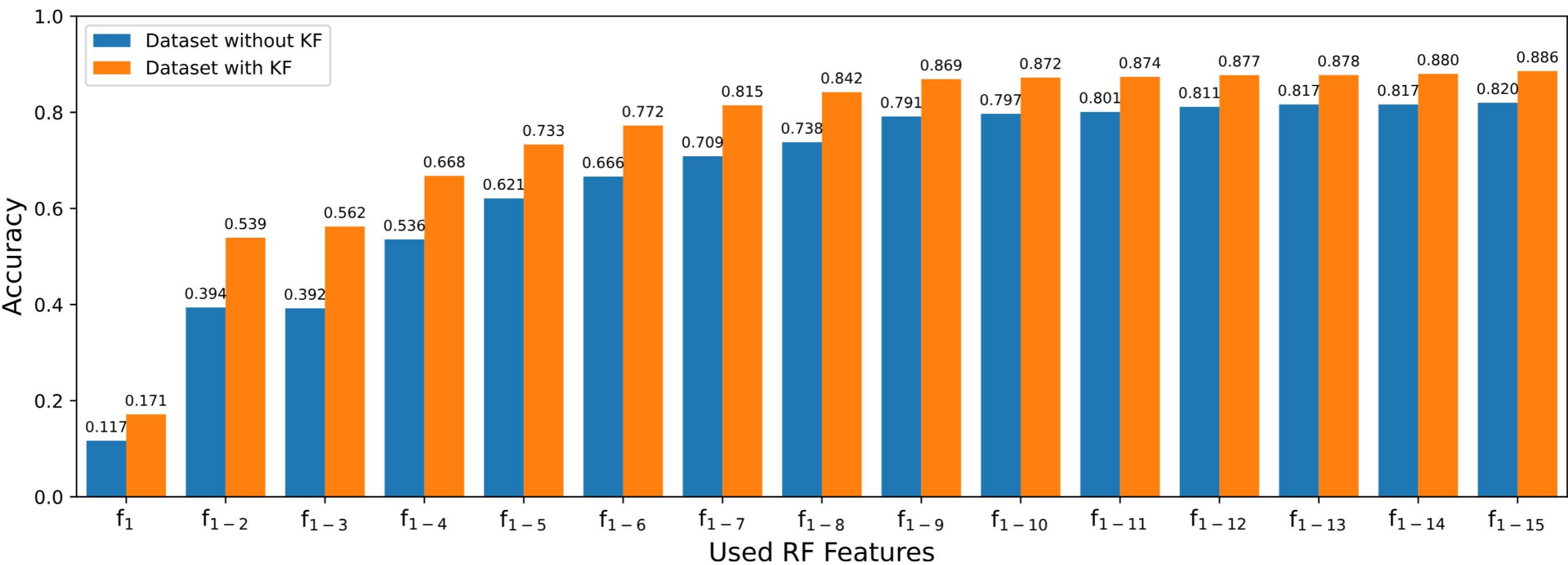
Feature importance for random forest model.

Notation	Feature	Importance
f_1	CFO	0.2199
f_2	Coarse CFO of STS (short_freq)	0.1760
f_3	Fine CFO of LTS (long_freq)	0.1442
f_4	Mean of phase error vector of L1 (phase_error_mean_1)	0.0562
f_5	Mean of phase error vector of L2 (phase_error_mean_2)	0.0544
f_6	Variance of phase error vector of L2 (phase_error_var_2)	0.0418
f_7	Variance of phase error vector of L1 (phase_error_var_1)	0.0417
f_8	I/Q gain imbalance of L1 (iqi_1)	0.0382
f_9	Mean of magnitude error vector of L2 (mag_error_mean_2)	0.0381
f_{10}	Mean of magnitude error vector of L1 (mag_error_mean_1)	0.0375
f_{11}	I/Q gain imbalance of L2 (iqi_2)	0.0375
f_{12}	Variance of magnitude error vector of L1 (mag_error_var_1)	0.0305
f_{13}	Variance of magnitude error vector of L2 (mag_error_var_2)	0.0305
f_{14}	Fractal dimension of L1 (frac_dimension_1)	0.0268
f_{15}	Fractal dimension of L2 (frac_dimension_2)	0.0265

Confusion Matrix



Identification accuracy versus feature groups



— CONTENTS —

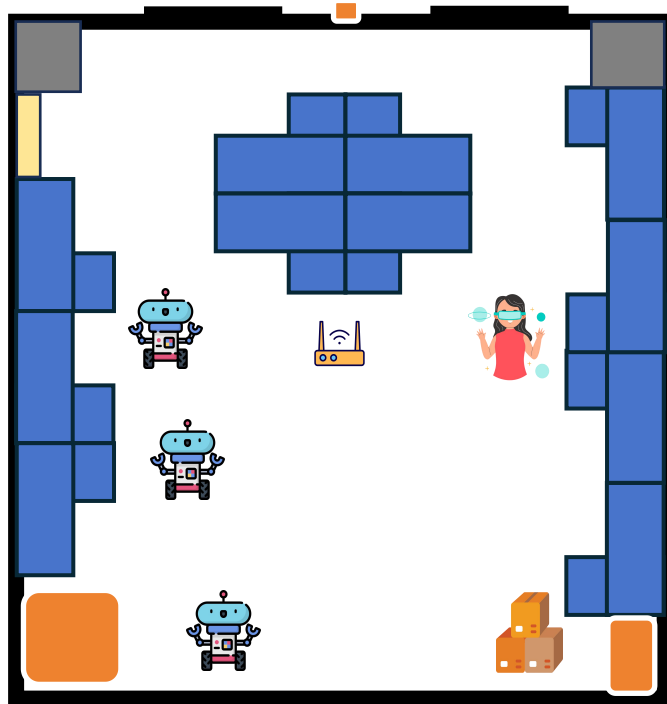
- 1. Background and Task Introduction
- 2. Signal and Feature Introduction
- 3. Data Acquisition Pipeline Walkthrough
- 4. Lab 1: Load and Explore the Dataset
- 5. Lab 2: Construct and Visualize RF Features
- 6. Lab 3: Build and Evaluate the Baseline
- **7. Summary and Open Discussion**

Challenges and Future Direction in RF Fingerprinting

Robustness Across Time, Channel, Environment, and Receiver



Device aging



Environment changing



Doppler effect, mobility fading



Receiver changing

Challenges and Future Direction in RF Fingerprinting

Scalability and Open-World Operation



Differences become smaller
Features overlap



Fine-grained features
Advanced deep learning algorithms

Open-set recognition

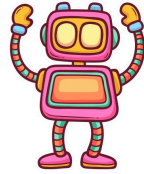
Unknown-device rejection

Few-shot enrollment

Continual learning

Challenges and Future Direction in RF Fingerprinting

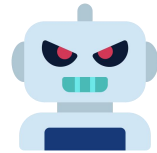
Security Under Adaptive RF Adversaries



Replay fingerprint emulation

Waveform manipulation

Adversarial perturbation



Backdoor attack

Thanks for your attending!

Q&A